

CMPE 260 Project - Extended BNF

Mustafa Şahin - 2023400162
Azra Zülal Muhcu - 2024400372

June 2026

Below is the Extended Backus-Naur Form (EBNF) grammar for the programming language. This grammar defines the operator precedence and associativity rules.

```
<program>          ::= <block>

<block>            ::= { <statement> ";" } [ <expr> ]

<statement>        ::= <let_stmt>
                       | <assign_stmt>
                       | <print_stmt>
                       | <expr_stmt>

<let_stmt>         ::= "let" <identifier> "=" <expr>

<assign_stmt>      ::= <identifier> "=" <expr>

<print_stmt>       ::= "print" "(" <expr> ")"

<expr_stmt>        ::= <expr>

<expr>             ::= <or_expr>

<or_expr>          ::= <and_expr>
                       { "or" <and_expr> }

<and_expr>         ::= <comparison_expr>
                       { "and" <comparison_expr> }

<comparison_expr>  ::= <add_expr> [ <comp_op> <add_expr> ]

<comp_op>          ::= "==" | "!=" | "<" | ">" | "<=" | ">="

<add_expr>         ::= <mult_expr> { <add_op> <mult_expr> }
```

```

<add_op>          ::= "+" | "-"

<mult_expr>       ::= <unary_expr> { <mult_op> <unary_expr> }

<mult_op>         ::= "*" | "/"

<unary_expr>      ::= { "not" | "-" }
                   <call_expr>

<call_expr>       ::= <primary_expr>
                   { "(" [ <arguments> ] ")" }

<arguments>       ::= <expr>
                   { "," <expr> }

<primary_expr>    ::= <integer>
                   | <boolean>
                   | <identifier>
                   | "(" <expr> ")"
                   | <if_expr>
                   | <fun_expr>

<if_expr>         ::= "if"
                   <expr>
                   "then"
                   <block>
                   "else"
                   <block>
                   "end"

<fun_expr>        ::= "fun"
                   "(" [ <params> ] ")"
                   "->"
                   <block>
                   "end"

<params>          ::= <identifier>
                   { "," <identifier> }

<boolean>         ::= "true"
                   | "false"

<integer>         ::= <digit>
                   { <digit> }

```

```
<identifier>      ::= <letter>
                    { <letter> | <digit> | "_" }

<letter>           ::= "a" | ... | "z"
                    | "A" | ... | "Z"

<digit>            ::= "0" | "1" | "2" | "3" | "4"
                    | "5" | "6" | "7" | "8" | "9"
```